

Proyecto 3

Integrantes del equipo:

Isabella Cadavid Posada
Isabella Ocampo Sánchez

Profesor:

José Luis Montoya Pareja

Materia:

Organización de computadores

Universidad EAFIT
Escuela de Ciencias Aplicadas e Ingeniería
Medellín
2026

Tabla de Contenido

Videos explicativos.....	3
Descripción General.....	4
Objetivo.....	5
Contexto: Conexión con el Proyecto 2.....	6
Decisiones Clave de Diseño.....	7
Videos Explicativos.....	8

Videos explicativos

- **Assembler:** <https://youtu.be/fJaLFzgwRSM?si=hrCuOb1CnUKHJJp3>
- **Disassembler:** <https://www.youtube.com/watch?v=0iYU1GuGSZo>
- **Prueba 1:** <https://www.youtube.com/watch?v=iGbeoPNbOyE>
- **Pruebas 2:** <https://youtu.be/1JCqFeL09ok?si=3Hs3RjXA8j2LLfJo>

Descripción General

El Proyecto 3 consiste en el diseño e implementación de un ensamblador y desensamblador completo para la arquitectura Hack de Nand2Tetris, desarrollado en Java. El sistema traduce programas escritos en lenguaje ensamblador (.asm) a código binario de 16 bits (.hack), y también realiza el proceso inverso. A diferencia del traductor estándar de Nand2Tetris, esta implementación incorpora soporte para las instrucciones de desplazamiento (<< shift left y >> shift right) definidas en la especificación extendida de EAFIT, las cuales fueron diseñadas e implementadas desde el Proyecto 2.

Objetivo

Construir una solución de software que sea capaz de:

- Leer un archivo .asm con instrucciones tipo A, tipo C, instrucciones Shift, etiquetas y comentarios.
- Traducirlo completamente a un archivo .hack con una instrucción binaria de 16 bits por línea.
- Desensamblar un archivo .hack de vuelta a mnemónicos ensamblador mediante el flag -d.
- Reportar errores con número de línea exacto y eliminar el archivo de salida parcial si ocurre alguno.

Contexto: Conexión con el Proyecto 2

En el Proyecto 2 se diseñaron e implementaron en HDL el chip Shifter y la ALU extendida con soporte para desplazamiento. También se definió la codificación binaria de las instrucciones Shift. El Proyecto 3 da continuidad a ese trabajo: ahora el ensamblador reconoce esas instrucciones en texto y las convierte al mismo binario que la CPU construida puede ejecutar, cerrando el ciclo completo de la arquitectura.

Decisiones Clave de Diseño

El sistema aplica el Principio de Responsabilidad Única (SRP), dividiendo la lógica en seis clases independientes. *La clase HackAssembler* actúa como punto de entrada y gestiona los argumentos CLI y el menú interactivo. *Assembler* orquesta las dos pasadas y maneja la lectura y escritura de archivos. *Parser* limpia las líneas, clasifica las instrucciones y extrae los campos semánticos. *CodeGenerator* traduce las instrucciones A, C y Shift a binario de 16 bits. *SymbolTable* mantiene el mapa símbolo → dirección y viene precargada con los símbolos predefinidos de Hack. Finalmente, *HackDisassembler* invierte el proceso completo, convirtiendo binario de vuelta a mnemónicos ensamblador.

Se utilizó el patrón de dos pasadas para resolver referencias hacia adelante: la primera pasada recorre el archivo identificando etiquetas y construyendo la tabla de símbolos, y la segunda genera el código binario con todos los símbolos ya resueltos.

La codificación de las instrucciones Shift ocupa el espacio de opcodes bit15:14 = 10, que no colisiona con instrucciones A (0xx) ni instrucciones C estándar (111). El bit 13 distingue la dirección: 1 para left, 0 para right.

Tipos de Instrucción Soportados

El sistema reconoce cuatro tipos de instrucción. Las instrucciones tipo A, como @42 o @suma, representan instrucciones de dirección. Las instrucciones tipo C, como D=M+1;JGT, corresponden a instrucciones de cómputo estándar. Las instrucciones Shift, como D=D<< o A=M>>.. Por último, las instrucciones tipo L, como (LOOP), son declaraciones de etiqueta que no generan código binario.

Videos Explicativos

A continuación se presentan los videos de justificación del proyecto, donde se expone el funcionamiento del sistema, las decisiones de diseño y la validación mediante casos de prueba.

Video 1 – Ensamblador y arquitectura del sistema

LINK: <https://youtu.be/fJaLFzgwRSM?si=hrCuOb1CnUKHJJp3>

Este video cubre la arquitectura general del HackAssembler.

Video 2 – Desensamblador y pruebas

LINK: <https://www.youtube.com/watch?v=0iYU1GuGSZo>

Este video presenta el funcionamiento del HackDisassembler, la lógica de identificación de instrucciones desde binario, el manejo de errores del sistema y la ejecución completa.

Pruebas

Link 1:

<https://www.youtube.com/watch?v=iGbeoPNbOyE>

Link 2:

<https://www.youtube.com/watch?si=3Hs3RjXA8j2LLfJo&v=1JCqFeL09ok&feature=youtu.be>

